Lembar Kerja Praktikum P07 – Exception Handling

Lembar kerja ini menggantikan laporan praktikum. Isi **sambil** mengerjakan setiap langkah di modul, lalu kumpulkan di akhir sesi sebagai `LK-P07-<NIM>.pdf` bersama zip kode Anda.

Petunjuk pengisian

- Jawab singkat dan jelas. Satu sampai tiga kalimat per pertanyaan sudah cukup.
- Tempel output program sebagai **teks**, bukan tangkapan layar.
- Di L2 dan L3, isi kolom **prediksi** sebelum menjalankan program. Prediksi yang salah tidak mengurangi nilai dan yang dinilai adalah penjelasan selisihnya.
- Total nilai LK 50 poin. Bagian L0 wajib diisi, tanpa poin.

L0 – Identitas dan parameter (wajib)

Isian	Jawaban
Nama	Az Zahrand Solichul Tajussalathin
NIM	2511532001
Kelas	A
Tanggal praktikum	12 November 2026
Versi Java (baris pertama `java -version`)	java version "25.0.5.0.2" 2026-10-21 LTS Pengujian terbaru memakai JDK 25 tanpa --enable-preview.
Saldo awal e-wallet (`NimParams`)	Rp200000
Stok awal KOPI-001	3
Stok awal RDG-002	1

L1 – Membaca crash dan menangani input (8 poin)

(a) Bedah stack trace `CrashDemo` skenario 2 dan 3 (4 poin). Contoh pengisian untuk skenario 1 ada di modul.

Yang dicari	Skenario 2	Skenario 3
Jenis exception	NumberFormatException	NullPointerException
Checked atau unchecked?	Unchecked	Unchecked
Pesan	For input string: "dua"	Cannot invoke "shop.model.Address.getCity()" because the return value of "shop.demo.CrashDemo$Customer.getAddress()" is null
Baris kode Anda yang pertama (file:baris, method)	CrashDemo.java:28, readQuantity	CrashDemo.java:32, shippingCity
Dipanggil dari (file:baris, method)	CrashDemo.java:43, main	CrashDemo.java:44, main

Yang dicari	Skenario 2	Skenario 3
Apa penyebabnya, dengan kata-kata Anda sendiri?	Teks "dua" dikonversi dengan Integer.parseInt, sehingga gagal dibaca sebagai bilangan bulat.	Alamat Customer belum diisi. getAddress() mengembalikan null, lalu kode memanggil getCity() pada nilai tersebut.

(b) Jumlah `-2` (2 poin). Sebelum `ShopApp` diperbaiki, apa yang terjadi saat Anda memilih produk 2 lalu mengetik jumlah `-2`? Mengapa tidak muncul `NumberFormatException`? Apa beda "format input salah" dengan "aturan bisnis dilanggar"?

Jawaban: Jumlah -2 berhasil dibaca sebagai int. Perulangan penambahan barang langsung dilewati karena i = 0 tidak memenuhi i < -2, sehingga keranjang tidak bertambah. NumberFormatException tidak muncul karena format -2 sah. Aturan jumlah positif harus diperiksa tersendiri dengan if.

(c) `if` atau `catch`? (2 poin). Mengapa nomor produk di luar katalog dicegah dengan `if`, sedangkan teks yang bukan angka ditangkap dengan `catch (NumberFormatException _)`?

Jawaban: Setelah teks berhasil dibaca sebagai int, batas nomor produk dapat diperiksa dengan if sebelum catalog.get dipanggil. Teks seperti abc gagal saat Integer.parseInt dijalankan, sehingga NumberFormatException ditangkap agar pengguna bisa mengisi ulang.

L2 – Prediksi dulu, baru jalankan (12 poin)

Isi Catatan pengisian: perkiraan pada LK ini disusun setelah pengujian berdasarkan pembacaan kode.kolom **Prediksi** sebelum menjalankan program. Tulis baris-baris output sesuai urutan, misalnya `A1, A2, A4, A5`.

`FlowDemo` bagian A, B, C (7 poin: A 2, B 3, C 2)

Bagian	Prediksi output	Output sebenarnya	Cocok?	Penjelasan selisih (atau mengapa tebakan Anda benar)
A: `partA("12")` lalu `partA("12rb")`	A1 mulaiA2 jumlah = 12A4 finallyA5 lanjutA1 mulaiA3 bukan angkaA4 finallyA5 lanjut	A1 mulai A2 jumlah = 12 A4 finally A5 lanjut A1 mulai A3 bukan angka A4 finally A5 lanjut	Cocok dengan perkiraan dari kode.	Input 12 berhasil dikonversi. Input 12rb masuk catch. finally dan baris sesudahnya tetap dijalankan pada kedua pemanggilan.
B: `partB(300000, "3")` lalu `partB(300000, "0")`	B1 bulan = 3B2 cicilan = 100000B4 finallyB5 hasil = 100000B1 bulan = 0B4 finallyB7 main menangkap: / by zero	B1 bulan = 3 B2 cicilan = 100000 B4 finally B5 hasil = 100000 B1 bulan = 0 B4 finally B7 main menangkap: / by zero	Cocok dengan perkiraan dari kode.	Pembagian dengan nol melempar ArithmeticException, yang ditangkap di main sesudah finally partB. Baris B6 dan pencetakan B5 kedua dilewati.
C: `partC(5)`	C1 cek stokC2 finallyC3 sisa = 4	C1 cek stok C2 finally C3 sisa = 4	Cocok dengan perkiraan dari kode.	Nilai return 4 dihitung di try, lalu finally dicetak sebelum nilai itu diterima main.

`CariProdukDemo` bagian D (3 poin)

Perintah	Prediksi output	Output sebenarnya	Penjelasan: catch mana yang jalan, dan mengapa
`...CariProdukDemo 2`	D1 SKU: RDG-002D5 pencarian selesai	D1 SKU: RDG-002 D5 pencarian selesai	Tidak masuk catch karena indeks 1 valid. finally tetap dijalankan.
`...CariProdukDemo abc`	D2 nomor harus bilangan bulatD5 pencarian selesai	D2 nomor harus bilangan bulat D5 pencarian selesai	catch NumberFormatException menangkap kegagalan parse abc.
`...CariProdukDemo 8`	D3 nomor di luar katalog: Index 7 out of bounds for length 3D5 pencarian selesai	D3 nomor di luar katalog: Index 7 out of bounds for length 3 D5 pencarian selesai	catch ArrayIndexOutOfBoundsException menangkap akses indeks 7 pada array berukuran 3.
`...CariProdukDemo` (tanpa argumen)	D4 Berikan satu nomor produk.D5 pencarian selesai	D4 Berikan satu nomor produk. D5 pencarian selesai	catch IllegalArgumentException menangkap exception yang dilempar saat args kosong.

L2.c – Ubah urutan catch (2 poin). Tempel pesan error compiler setelah catch `IllegalArgumentException` dipindah ke paling atas, lalu jelaskan artinya dalam satu kalimat.

```
Pesan error:
output\lk-evidence\catch-check\CariProdukDemo.java:16: error: exception NumberFormatException has already been caught
        } catch (NumberFormatException e) {
          ^
1 error
```

Artinya: NumberFormatException adalah turunan IllegalArgumentException, sehingga catch induk di atas sudah menangkapnya dan catch khusus di bawah tidak dapat dicapai.

L3 – Custom exception dan checkout (10 poin)

(a) Error berantai (2 poin). Setelah `PaymentMethod.pay` diubah menjadi `void ... throws PaymentFailedException`, berapa error yang muncul, dan di file mana saja? Mengapa satu perubahan di interface memengaruhi file-file itu?

Jumlah error: BELUM DIISI. File: BELUM DIISI.
Catatan: kondisi tepat sesudah interface diubah belum direkam. Isi dari catatan atau hasil kompilasi pada tahap L3.c, jangan memakai jumlah perkiraan modul.

Alasan: Semua implementasi PaymentMethod harus mengikuti tipe hasil dan deklarasi exception yang baru. Pemanggil pay juga harus menyesuaikan penggunaan nilai hasil serta menangkap atau meneruskan checked exception PaymentFailedException.

(b) Enam skenario `ScenarioRunner` (4 poin). Isi kolom prediksi dulu (pakai parameter L0), baru jalankan. Hasil diisi salah satu dari `[OK]`, `[SALDO]`, `[BAYAR]`, `[STOK]`, `[INVALID]`.

No.	Prediksi hasil	Prediksi sesudahnya: KOPI / RDG / TEH / saldo	Hasil sebenarnya	Sebenarnya: KOPI / RDG / TEH / saldo	Cocok?
S1 Kopi + Rendang, Reguler, e-wallet	[SALDO]	3 / 1 / 5 / Rp200000	[SALDO]	3 / 1 / 5 / Rp200000	Cocok

No.	Prediksi hasil	Prediksi sesudahnya: KOPI / RDG / TEH / saldo	Hasil sebenarnya	Sebenarnya: KOPI / RDG / TEH / saldo	Cocok?
S2 Kopi x3, Reguler, transfer bank	[BAYAR]	3 / 1 / 5 / Rp200000	[BAYAR]	3 / 1 / 5 / Rp200000	Cocok
S3 Rendang x2, Reguler, e-wallet	[STOK]	3 / 1 / 5 / Rp200000	[STOK]	3 / 1 / 5 / Rp200000	Cocok
S4 Kopi, Ekspres, kartu kredit	[SALDO]	3 / 1 / 5 / Rp200000	[SALDO]	3 / 1 / 5 / Rp200000	Cocok
S5 Keranjang kosong	[INVALID]	3 / 1 / 5 / Rp200000	[INVALID]	3 / 1 / 5 / Rp200000	Cocok
S6 Teh x2, Reguler, e-wallet	[OK]	3 / 1 / 3 / Rp120000	[OK]	3 / 1 / 3 / Rp120000	Cocok

Pilih satu skenario yang prediksinya meleset (atau, jika semua cocok, skenario yang paling menarik), lalu jelaskan mengapa hasilnya seperti itu. Sebut juga mengapa pesan `[STOK]` menyebut "diminta 1".

Jawaban: Pada S3, Rendang pertama berhasil dipesan sehingga stok sementara menjadi 0. Pemesanan Rendang kedua gagal dan seluruh stok yang sudah dipesan dikembalikan, sehingga stok akhirnya tetap 1. Pesan menyebut "diminta 1" karena checkout memanggil reserve(sku, 1) untuk setiap barang di keranjang.

(c) Mengembalikan stok lalu `throw e` (2 poin). Di method `checkout`, mengapa blok catch mengembalikan stok lalu melempar ulang exception yang sama, bukan mencetak pesan untuk pembeli? Apa yang terjadi pada stok jika `release` dihapus?

Jawaban: Checkout mengembalikan barang yang sudah dipesan agar transaksi gagal tidak mengurangi stok. throw e meneruskan jenis exception dan datanya ke ShopApp.buy, tempat pesan pembeli dibuat. Jika release dihapus, stok yang sempat dipesan tetap berkurang meskipun pembayaran atau pemesanan berikutnya gagal.

(d) Urutan catch di `buy` (2 poin). Mengapa catch `InsufficientBalanceException` harus ditulis di atas catch `PaymentFailedException`? Apa yang terjadi jika urutannya ditukar?

Jawaban: InsufficientBalanceException adalah turunan PaymentFailedException, sehingga catch khusus saldo harus lebih dulu. Jika ditukar, compiler menolak catch InsufficientBalanceException karena exception itu sudah ditangkap oleh catch PaymentFailedException.

L4 – Katalog dari file (6 poin)

(a) Baris katalog yang dilewati (3 poin).

Baris ke-	Isi baris	Apa masalahnya?	Exception yang tertangkap (nama class)
3	Field: KUE-003 | Kue Lapis Legit | 95rb | 800	Harga 95rb gagal dibaca sebagai long.	NumberFormatException (ditangkap catch IllegalArgumentException)
5	Field: BATIK-005 | Kain Batik Tulis	Hanya ada 2 field. Diperlukan 4 field.	IllegalArgumentException
6	Field: SAMB-006 | Sambal Lado Mudo 200g | -45000 | 250	Harga negatif ditolak oleh konstruktor produk.	IllegalArgumentException

(b) Ketika file tidak ada (3 poin). Tempel output `java -cp bin shop.app.ShopApp katalog-cadangan.csv`. Apa isi `e.getMessage()` dan `e.getCause()`? Kalau `ShopApp` hanya mencetak `e.getCause().getMessage()`, apa yang dilihat pengguna, dan mengapa itu kurang membantu?

```
Output:
Toko tidak bisa dibuka: Katalog gagal dibaca: katalog-cadangan.csv
Penyebab asli: java.nio.file.NoSuchFileException: katalog-cadangan.csv
```

Jawaban: e.getMessage() berisi "Katalog gagal dibaca: katalog-cadangan.csv". e.getCause() adalah java.nio.file.NoSuchFileException: katalog-cadangan.csv. Jika hanya cause.getMessage() dicetak, pengguna hanya melihat nama file tanpa keterangan bahwa katalog gagal dibaca.

L5 – Code review FragileShop (10 poin)

Tabel temuan (8 poin, 2 per method).

Method	Apa masalahnya?	Dampaknya (bagi pembeli atau programmer)	Perbaikan
`payOrder`	Kegagalan pembayaran ditangkap oleh catch kosong.	Pemanggil mengira pembayaran selesai meskipun saldo kurang.	Hapus catch kosong, deklarasikan throws PaymentFailedException, lalu tangani kegagalan di main.
`isInStock`	Pengecekan ketersediaan memakai reserve dan release.	Pengecekan memiliki efek samping sementara pada stok dan memakai exception untuk kondisi ya/tidak.	Periksa inventory.available(sku) > 0.
`readCatalogHeader`	Cause diganti null saat IOException dibungkus.	Penyebab kegagalan file hilang dari rantai exception.	Teruskan e ke new CatalogLoadException(message, e).
`addToCart`	Kesalahan input dan stok habis digabung dalam satu multi-catch.	Pembeli menerima pesan teknis dan sulit membedakan input keliru dari stok kurang.	Tangani NumberFormatException secara khusus, validasi jumlah positif, lalu tangani OutOfStockException dengan pesan stok.

Output `FragileShop` setelah `payOrder` dan `readCatalogHeader` diperbaiki (2 poin).

```
Output:
1) Pembayaran gagal: Saldo tidak cukup: perlu Rp85000, tersedia Rp50000
2) Kopi tersedia? true
3) Gagal: jumlah harus berupa angka.
   Gagal: stok KOPI-001 tidak cukup, tersedia 1.
4) Katalog gagal dibaca: katalog-lama.csv | penyebab: java.nio.file.NoSuchFileException: katalog-lama.csv
```

Penutup

Hasil SelfCheck terakhir. Salin dua baris berikut dari output SelfCheck.

```
=== HASIL: 24 / 24 pemeriksaan lulus (nilai SelfCheck 50 / 50) ===
Seluruh pemeriksaan wajib lulus pada Java 25.
```

Tidak ada baris === TANTANGAN pada output. Pemeriksaan bonus masih dikomentari dalam SelfCheck.

Refleksi (4 poin).

1. (2 poin) Mengapa kesalahan ketik pembeli (misalnya `dua` sebagai jumlah) perlu ditangani berbeda dari stok yang habis? Hubungkan dengan pilihan checked/unchecked.

Jawaban: Kesalahan ketik dapat diperbaiki dengan meminta input ulang setelah NumberFormatException ditangkap. Exception ini unchecked karena berasal dari konversi teks. Stok habis adalah kegagalan bisnis yang dilaporkan lewat checked exception OutOfStockException, sehingga pemanggil reserve wajib menangkap atau meneruskannya.

2. (1 poin) Apakah pesan exception teknis, misalnya `For input string: "dua"`, sebaiknya ditampilkan langsung kepada pembeli? Beri satu contoh dari praktikum ini.

Jawaban: Pesan teknis perlu diubah ke bahasa yang dipahami pembeli. Saat jumlah diisi dua, aplikasi dapat mencetak "Input harus berupa angka." dan meminta input ulang. Detail For input string: "dua" lebih berguna untuk pemeriksaan kode.

3. (1 poin, dinilai penuh asal diisi jujur) Satu hal tentang exception yang masih membingungkan atau ingin Anda tanyakan di pertemuan berikutnya.

BELUM DIISI - tulis satu hal yang masih membingungkan atau ingin ditanyakan berdasarkan pengalaman sendiri.